

Face Recognition via Eigenfaces

A Mathematical and Code Walkthrough

Aditya Maknikar

Machine Learning from Scratch

Mathematical Background: pg 1–4

Code Walkthrough: pg 5–11

1 Mathematical Background

1.1 How Images Become Numbers

Before any math can happen, the computer needs to turn a picture into something it can actually work with. A grayscale image is stored as a rectangular grid of pixels, where each pixel holds a single number representing brightness — 0 is black, 255 is white, and everything in between is a shade of grey.

Think of it like a spreadsheet: every row is a row of pixels, every column is a column of pixels, and every cell contains a brightness value. A 112-by-92-pixel image is literally just a 112×92 spreadsheet of numbers.

In the code, once an image is loaded it gets *flattened* — the 2D grid is unrolled into a single long row of $L = 112 \times 92 = 10304$ numbers. One image becomes one row in a larger matrix.

1.2 The Dataset Matrix A

We have M face images (after holding some out for testing, $M = 389$). After flattening each image, we stack them on top of each other to form a matrix A . This matrix has shape $M \times L$: each row is one image, each column is one pixel position.

$$A \in \mathbb{R}^{M \times L}, \quad M = 389, \quad L = 10304$$

Each row as one person's face, compressed into a single long list of brightness values

1.3 Why We Subtract the Mean

Before doing any analysis, we subtract the mean face from every image in the dataset.

The mean vector $\mathbf{a}$ is computed by averaging across all M images, pixel by pixel. The result is a single vector of length L — the “average face.” We then subtract it from every row of A to produce the mean-centred matrix C :

$$C = A - \mathbf{a}, \quad \text{where } \mathbf{a} = \frac{1}{M} \sum_{i=1}^M A_i$$

We do this because we do not want our analysis to be dominated by the fact that all faces share a common baseline of brightness and structure. If we skip mean subtraction, the first principal component ends up describing “faces in general” rather than “the directions in which faces vary from each other.” By subtracting the mean we force the analysis to focus on differences like the variation in expression, lighting, pose, etc. that actually makes one face distinct from another.

In the code this step is handled implicitly by `sklearn`’s PCA, which stores the mean internally as `pca.mean_` and subtracts it for us. But conceptually this is the first thing that happens.

1.4 The Covariance Matrix and Why It Is Enormous

Once we have C , we want to understand the structure of the variation in the data. The tool for that is the covariance matrix S :

$$S = C^\top C$$

Think of S as a measure of how much any two pixel positions tend to vary together across all the images. If pixel 500 and pixel 600 are both bright or both dark in most images, their covariance is high. If one is bright when the other is dark, the covariance is low (or negative). A problem we face is S has shape $L \times L$, which means it is 10304×10304 . That is over 100 million entries. Computing the eigenvectors of a matrix that large is not practical on ordinary hardware.

1.5 The $M \times M$ Trick

Instead of finding the eigenvectors of the $L \times L$ matrix $S = C^\top C$, we can instead work with the much smaller $M \times M$ matrix:

$$S' = C \cdot C^\top$$

S' has shape $M \times M = 389 \times 389$. Finding eigenvectors of *this* matrix is fast.

If $\mathbf{v}$ is an eigenvector of $CC^\top$ (the small matrix), then $C^\top \mathbf{v}$ is an eigenvector of $C^\top C$ (the big matrix), with the same eigenvalue. The two matrices share the same non-zero eigenvalues, and their eigenvectors are directly related. So we can compute eigenvectors of the small matrix and transform them into eigenvectors of the large one.

Stated formally: if $CC^\top \mathbf{v} = \lambda \mathbf{v}$, then:

$$C^\top C (C^\top \mathbf{v}) = C^\top (CC^\top \mathbf{v}) = C^\top (\lambda \mathbf{v}) = \lambda (C^\top \mathbf{v})$$

So $\mathbf{u} = C^\top \mathbf{v}$ is an eigenvector of $C^\top C$ with eigenvalue λ . In practice, we then normalize $\mathbf{u}$ so it has unit length. Scikit-learn handles this internally. when we call `PCA().fit()`, this is the shortcut it uses. It does not compute $C^\top C$ directly; it works with the smaller matrix to make the computation tractable.

1.6 Principal Components as Eigenfaces

The eigenvectors of $S = C^\top C$ are the *principal components* of the face dataset. Because each eigenvector has length $L = 10304$, it can be reshaped back into a 112×92 image. When you do that, the resulting images look like blurry, ghostly faces. These are the eigenfaces.

The eigenvalues associated with each eigenvector tell us how much of the variation in the dataset that eigenvector explains. The eigenvectors are ordered from highest eigenvalue to lowest so the first eigenface captures the single most important direction of variation, the second captures the next most important, and so on.

We keep only the top K eigenfaces (in the code, $K = 50$). The remaining ones are discarded because they explain a very small fraction of the total variation. Formally, we collect the top K eigenvectors as rows of a matrix F of shape $K \times L$.

1.7 Representing Any Face as a Weight Vector

Any face image can be expressed as a weighted sum of the eigenfaces, plus the mean face. Eigen faces are similar to primary colours and the weights say how much of each colour to use.

Formally, for a face image $\mathbf{x}$ (a row vector of length L), we first subtract the mean to get the centred version $\tilde{\mathbf{x}} = \mathbf{x} - \mathbf{a}$. We then project this onto each eigenface by computing the dot product:

$$w_j = \mathbf{f}_j \cdot \tilde{\mathbf{x}}^\top, \quad j = 1, \dots, K$$

where $\mathbf{f}_j$ is the j -th eigenface (a row of F). The result is a vector of K weights:

$$\mathbf{w} = F \cdot \tilde{\mathbf{x}}^\top \in \mathbb{R}^K$$

This weight vector is a compact representation of the face. Two different pictures of the same person should produce similar weight vectors, even if the photos are not identical. A picture of a completely different person should produce a noticeably different weight vector.

1.8 Face Matching via Euclidean Distance

To recognise a face, we compare its weight vector to the weight vectors of all known faces and find the closest one. The distance metric used is the Euclidean (L2) distance:

$$d(\mathbf{w}_1, \mathbf{w}_2) = \|\mathbf{w}_1 - \mathbf{w}_2\|_2 = \sqrt{\sum_{j=1}^K (w_{1j} - w_{2j})^2}$$

The known face with the smallest distance to the query is the best match. A threshold can optionally be applied: if the best match distance is above the threshold, the face is declared “unknown” — someone the model has never seen.

2 Code Walkthrough

The following walkthrough maps every block of code to the mathematical concepts above. Each code block is shown alongside the explanation of what it is actually doing.

2.1 Loading the Images

```
1 import cv2
2 import zipfile
3 import numpy as np
4
5 faces = {}
6 with zipfile.ZipFile("attface.zip") as facezip:
7     for filename in facezip.namelist():
8         if not filename.endswith(".pgm"):
9             continue # not a face picture
10        with facezip.open(filename) as image:
11            faces[filename] = cv2.imdecode(
12                np.frombuffer(image.read(), np.uint8),
13                cv2.IMREAD_GRAYSCALE
14            )
```

Listing 1: Reading face images from a zip archive

The dataset is the AT&T Database of Faces: 40 people, 10 photos each, 400 images total. Each image is stored as a PGM file (grayscale, no colour channels). The code iterates through the zip archive, reads each PGM file into a byte buffer, and uses OpenCV's `cv2.imdecode` to turn that buffer into a 2D NumPy array of pixel values.

After this block, `faces` is a Python dictionary mapping each filename (e.g. "s1/1.pgm") to its corresponding 112×92 NumPy array. Nothing mathematical has happened yet — this is purely data ingestion.

2.2 Splitting into Training and Test Sets

- All 10 images of person 40 are held out. This person is entirely unknown to the model, so querying with their photo should produce a high distance (i.e. a failed recognition).
- One image of person 39 (s39/10.pgm) is held out. This person *is* in the model, so querying with their photo should produce a low distance and a correct match.

```

1 facematrix = []
2 facelabel  = []
3
4 for key, val in faces.items():
5     if key.startswith("s40/"):
6         continue          # hold out entire person 40
7     if key == "s39/10.pgm":
8         continue          # hold out one image from person 39
9     facematrix.append(val.flatten())
10    facelabel.append(key.split("/") [0])
11
12 facematrix = np.array(facematrix)

```

Listing 2: Building the training matrix

For every image that is kept, `val.flatten()` converts the 112×92 2D array into a 1D vector of length $L = 10304$. This is the vectorisation step described in Section 1.1. The final call `np.array(facematrix)` stacks all those 1D vectors into the $M \times L$ matrix A from Section 1.2.

The shape is therefore:

$$A \in \mathbb{R}^{389 \times 10304}$$

`facelabel` stores the class identifier for each row (e.g. "s1", "s2", ..., "s39") so we know who each row corresponds to when we look up a match later.

2.3 Computing the Eigenfaces with PCA

```

1 from sklearn.decomposition import PCA
2
3 pca = PCA().fit(facematrix)
4
5 n_components = 50
6 eigenfaces = pca.components_[:n_components]

```

Listing 3: Fitting PCA and extracting the top K eigenfaces

`PCA().fit(facematrix)` does the heavy lifting. Internally, `sklearn` performs the $M \times M$ trick described in Section 1.5 and returns the principal components sorted by decreasing explained variance. We treat this as a black box: we put our matrix A in, and we get out the eigenvectors of the covariance matrix $C^T C$, ordered from most to least important. `pca.components_` is an array where each row is one principal component vector one eigenface of length $L = 10304$. We keep only the first 50.

$$F = \text{eigenfaces} \in \mathbb{R}^{50 \times 10304}$$

`pca.mean_` stores the mean face vector **a** of length L . This is used in every subsequent computation to re-centre images before projecting them.

The choice of $K = 50$ is a practical compromise. The explained variance array printed from `pca.explained_variance_ratio_` shows that the first component alone explains about 17.8% of total variance, the second about 12.9%, and the numbers drop off quickly after that. By 50 components, we have captured the majority of the meaningful variation in the dataset without carrying along hundreds of noisy, low-importance components.

2.4 Visualising the Eigenfaces

```

1 import matplotlib.pyplot as plt
2
3 fig, axes = plt.subplots(4, 4, sharex=True, sharey=True, figsize
    =(8, 10))
4 for i in range(16):
5     axes[i % 4][i // 4].imshow(
6         eigenfaces[i].reshape(faceshape), cmap="gray"
7     )
8 plt.show()
```

Listing 4: Displaying the first 16 eigenfaces

Each eigenface is stored as a flat vector of length 10304. To display it as an image, `.reshape(faceshape)` converts it back into a 112×92 array. The eigenfaces look like blurry, ghostly face images because they are literally the directions of maximum variation in face-pixel space. The first few show broad lighting effects and general face structure. Further down the list they pick up finer details like expression and hair.

2.5 Computing Weight Vectors for All Training Faces

```

1 weights = eigenfaces @ (facematrix - pca.mean_).T
```

Listing 5: Projecting each training face onto the eigenface space

This single line of code implements the projection described in Section 1.6. Breaking it down step by step:

1. `facematrix - pca.mean_` subtracts the mean face `a` from every row of `A`, producing the centred matrix `C`. This is an element-wise broadcast operation: NumPy automatically subtracts the $1 \times L$ mean vector from each of the M rows.
2. `.T` transposes `C` from shape $M \times L$ to shape $L \times M$, so that each column is one mean-centred face.
3. `eigenfaces @` performs matrix multiplication between `F` (shape $K \times L$) and `CT` (shape $L \times M$). The `@` operator in Python is matrix multiplication (introduced in Python 3.5).

The result is:

$$W = F \cdot C^T \in \mathbb{R}^{K \times M} = \mathbb{R}^{50 \times 389}$$

Each column of W is the weight vector for one training face. The j -th entry in that column says how strongly the j -th eigenface contributes to that particular image. Alternatively, each row of W is the coefficient for one eigenface across all 389 training images.

This is equivalent to the explicit loop below, which does the same computation element-by-element but is much slower:

```

1 weights_loop = []
2 for i in range(facematrix.shape[0]):
3     weight = []
4     for j in range(n_components):
5         w = eigenfaces[j] @ (facematrix[i] - pca.mean_)
6         weight.append(w)
7     weights_loop.append(weight)

```

Listing 6: Equivalent loop form (slower, for illustration only)

The one-line matrix version and this loop produce identical results. The matrix form is preferred in practice because it runs much faster.

2.6 Recognising a Known Person

```
1 query = faces["s39/10.pgm"].reshape(1, -1)
2 query_weight = eigenfaces @ (query - pca.mean_).T
3
4 euclidean_distance = np.linalg.norm(weights - query_weight, axis
5     =0)
6
7 best_match          = np.argmin(euclidean_distance)
8
9 print("Best match %s with Euclidean distance %f"
10      % (facelabel[best_match], euclidean_distance[best_match]))
```

Listing 7: Query with a held-out image from a known class

The query image is person 39, photo 10 — held out from the training set but belonging to a person who *is* in the model.

`query.reshape(1, -1)` converts the 112×92 image into a 1×10304 row vector, matching the shape of the training rows in `facematrix`.

`eigenfaces @ (query - pca.mean_).T` repeats the projection from Section 2.5, this time for just the one query image. The result `query_weight` has shape $K \times 1$ — the weight vector for the query.

`weights - query_weight` broadcasts the subtraction across all 389 columns of `weights`. `np.linalg.norm(..., axis=0)` then computes the Euclidean L2 distance between the query weight vector and each of the 389 training weight vectors:

$$d_i = \|\mathbf{w}_i - \mathbf{w}_{\text{query}}\|_2, \quad i = 1, \dots, 389$$

`np.argmin` returns the index of the smallest distance. The result is:

```
Best match s39 with Euclidean distance 1559.997137
```

The model correctly identifies person 39. The relatively low distance (compared to the unknown person test below) confirms that the query image sits close to the other images of person 39 in the eigenface weight space.

2.7 Attempting to Recognise an Unknown Person

```
1 query = faces["s40/1.pgm"].reshape(1, -1)
2 query_weight = eigenfaces @ (query - pca.mean_).T
3
4 euclidean_distance = np.linalg.norm(weights - query_weight, axis
   =0)
5 best_match          = np.argmin(euclidean_distance)
6
7 print("Best match %s with Euclidean distance %f"
8       % (facelabel[best_match], euclidean_distance[best_match]))
```

Listing 8: Query with an image from a class never seen during training

This block is identical to the previous one, but the query image is person 40 — someone the model has never seen. The model cannot return the correct answer because person 40’s weight vector simply is not in the training set. The result is:

Best match s5 with Euclidean distance 2690.209330

Two things to note. First, the best-match distance is noticeably higher (2690 vs. 1560). This is the signal that tells us the query is likely an unknown person. Second, the model still returns *some* best match (s5), because `np.argmin` always returns the closest thing it can find, even if the closest is still quite far.

This motivates the threshold idea: in a real system, you would choose a cutoff value τ and say: if $d_{\min} < \tau$, report the best match; if $d_{\min} \geq \tau$, report “unknown.” Choosing τ well is an empirical problem that requires a validation set.

2.8 Generating a New Face from Random Weights

```
1 random_weights = np.random.randn(n_components) * weights.std()
2 newface = random_weights @ eigenfaces + pca.mean_
3
4 fig, axes = plt.subplots(1, 2, sharex=True, sharey=True, figsize
   =(8, 6))
5 axes[0].imshow(pca.mean_.reshape(faceshape), cmap="gray")
6 axes[0].set_title("Mean face")
7 axes[1].imshow(newface.reshape(faceshape), cmap="gray")
8 axes[1].set_title("Random face")
9 plt.show()
```

Listing 9: Constructing a face from randomly sampled weights

This block demonstrates the reconstruction direction: instead of going from image to weights, we go from weights to image.

`np.random.randn(n_components)` samples $K = 50$ values from a standard normal distribution. Multiplying by `weights.std()` scales them to roughly the same range as real face weights, so the result does not look completely unrecognisable.

`random_weights @ eigenfaces` computes a weighted sum of the eigenfaces:

$$\tilde{\mathbf{z}} = \sum_{j=1}^K w_j \mathbf{f}_j \in \mathbb{R}^L$$

Adding `pca.mean_` restores the mean face, giving the final reconstructed image:

$$\mathbf{z} = \tilde{\mathbf{z}} + \mathbf{a}$$

The result is a face-like image that does not correspond to any real person. It looks plausible because the eigenfaces encode real statistical structure from the training data — the result is constrained to the “face-shaped” region of pixel space that the PCA model has learned.

2.9 Accuracy and Limitations

The eigenface method is surprisingly effective given how simple the underlying math is. Testing under controlled conditions has found recognition rates around 96% when lighting is consistent, dropping to about 85% with changes in head orientation, and down to 64% when subject distance varies. The core problem is that PCA is a linear method operating on raw pixel values. Changes in lighting, scale, or pose produce large movements in pixel space that do not correspond to changes in identity. A zoom-in or zoom-out, for instance, shifts pixel values in a way the model has no way to factor out.